INDEX

S.NO		PAGE NO
1.	**Implement DFS and BFS Algorithms using python language.**	2
2.	**Implement A* Search algorithm.**	3
3.	**Implement AO* Search algorithm.**	4
4.	**For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm to output a description of the set of all hypotheses consistent with the training examples**	6
5.	**Write a program to demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample**	7
6.	**Build an Artificial Neural Network by implementing the Back propagation algorithm and test the same using appropriate data sets.**	8
7.	**Write a program to implement the naïve Bayesian classifier for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering few test data sets**	10
8.	**Apply EM algorithm to cluster a set of data stored in a .CSV file. Use the same data set for clustering using k-Means algorithm. Compare the results of these two algorithms and comment on the quality of clustering. You can add Java/Python ML library classes/API in the program**	12
9.	**Write a program to implement k-Nearest Neighbour algorithm to classify the iris data set. Print both correct and wrong predictions. Java/Python ML library classes can be used for this problem**	14
10.	**Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select appropriate data set for your experiment and draw graphs**	15
11.	**Write a program to implement data preprocessing and text cleaning using python.**	18
12.	**Write a program to demonstrate parts-of-speech tagging and named entity recognition**	20

1. Implement DFS and BFS Algorithms using python language.

CODE:

```
graph = {
  'A': ['B', 'C'],
  'B': ['D', 'E'],
  'C': ['F'],
  'D': [],
  'E': ['F'],
  'F': []
}
# DFS (Recursive)
def dfs_recursive(node, visited=None):
  if visited is None:
    visited = set()
  visited.add(node)
  print(node, end=" ")
  for neighbour in graph[node]:
    if neighbour not in visited:
      dfs_recursive(neighbour, visited)
# DFS (Iterative Using Stack)
def dfs_iterative(start):
  visited = set()
  stack = [start]
  print("\nDFS Iterative:", end=" ")
  while stack:
    node = stack.pop()
    if node not in visited:
      print(node, end=" ")
      visited.add(node)
      stack.extend(graph[node][::-1])
# BFS (Using Queue)
from collections import deque
def bfs(start):
  visited = set()
  queue = deque([start])
  print("\nBFS:", end=" ")
  while queue:
    node = queue.popleft()
    if node not in visited:
      print(node, end=" ")
      visited.add(node)
```

```
        queue.extend(graph[node])  # add neighbors
print("DFS Recursive:", end=" ")
dfs_recursive('A')
dfs_iterative('A')
bfs('A')
```

OUTPUT:

```
DFS Recursive: A B D E F C
DFS Iterative: A B D E F C
BFS: A B C D E F
```

---

2. Implement A* Search algorithm.

CODE:

```
import networkx as nx
G = nx.Graph()
print("Enter edges (from to weight), 'done' to finish:")
while True:
    edge = input()
    if edge.lower() == 'done': break
    src, dst, weight = edge.split()
    G.add_edge(src, dst, weight=int(weight))
print("\nEnter heuristic values for each node to the goal")
heuristic_values = {}
for node in G.nodes():
    h_value = float(input(f"Enter heuristic value for node {node}: "))
    heuristic_values[node] = h_value
start = input("\nStart node: ")
end = input("End node: ")
def calculate_path_costs(path):
    print("\nDetailed path analysis:")
    print("Node\tg(n)\th(n)\tf(n)")
    print("-" * 30)
    g_cost = 0
    for i in range(len(path)):
        node = path[i]
        h_cost = heuristic_values[node]
        f_cost = g_cost + h_cost
        print(f"{node}\t{g_cost}\t{h_cost}\t{f_cost}")
        if i < len(path) - 1:
            next_node = path[i + 1]
            g_cost += G[node][next_node]['weight']
def heuristic(node1: str, node2: str) -> float:
    return heuristic_values[node1]
try:
    path = nx.astar_path(G, start, end, heuristic=heuristic, weight='weight')
```

```
    cost = nx.astar_path_length(G, start, end, heuristic=heuristic, weight='weight')
    print(f"\nPath: {' -> '.join(path)}")
    print(f"Total Cost (g): {cost}")
    calculate_path_costs(path)
except nx.NetworkXNoPath:
    print("\nNo path found!")
```

OUTPUT:

```
Enter edges (from to weight), 'done' to finish:
 A B 1
 B C 2
 done

Enter heuristic values for each node to the goal
Enter heuristic value for node A:  2
Enter heuristic value for node B:  4
Enter heuristic value for node C:  2

Start node:  A
End node:  C

Path: A -> B -> C
Total Cost (g): 3

Detailed path analysis:
Node  g(n)    h(n)    f(n)
------------------------------
A       0       2.0     2.0
B       1       4.0     5.0
C       3       2.0     5.0
```

---

3. Implement AO* Search algorithm.

CODE:

```
import heapq

class AOStar:
    def __init__(self, graph, start, goal):
        self.graph = graph
        self.start = start
        self.goal = goal
        self.open_list = []
        self.came_from = {}

    def heuristic(self, current):
        return abs(self.goal[0] - current[0]) + abs(self.goal[1] - current[1])

    def a_star(self):
```

```
        heapq.heappush(self.open_list, (0 + self.heuristic(self.start), self.start))
        g_scores = {self.start: 0}
        f_scores = {self.start: self.heuristic(self.start)}
        while self.open_list:
            _, current = heapq.heappop(self.open_list)
            if current == self.goal:
                path = []
                total_cost = g_scores[current]
                while current in self.came_from:
                    path.append(current)
                    current = self.came_from[current]
                path.append(self.start)
                return path[::-1], total_cost

            for neighbor, cost in self.graph.get(current, {}).items():
                tentative_g_score = g_scores[current] + cost

                if neighbor not in g_scores or tentative_g_score < g_scores[neighbor]:
                    self.came_from[neighbor] = current
                    g_scores[neighbor] = tentative_g_score
                    f_scores[neighbor] = tentative_g_score + self.heuristic(neighbor)
                    heapq.heappush(self.open_list, (f_scores[neighbor], neighbor))
        return None, 0

if __name__ == "__main__":
    graph = {
        (0, 0): {(0, 1): 2, (1, 0): 1},
        (0, 1): {(0, 0): 2, (1, 1): 2},
        (1, 0): {(0, 0): 1, (1, 1): 1},
        (1, 1): {(0, 1): 2, (1, 0): 1, (2, 1): 3},
        (2, 1): {(1, 1): 3, (2, 0): 1},
        (2, 0): {(2, 1): 1}
    }

    start = (0, 0)
    goal = (2, 1)

    ao_star = AOStar(graph, start, goal)
    path, total_cost = ao_star.a_star()

    if path:
        print(f"Path found: {path}")
        print(f"Total cost: {total_cost}")
    else:
        print("No path found")
```

OUTPUT:

Path found: [(0, 0), (1, 0), (1, 1), (2, 1)]

Total cost: 5

4. For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm to output a description of the set of all hypotheses consistent with the training examples.

CODE:

```
import pandas as pd

def candidate_elimination(data):
    features = len(data.columns) - 1
    G = [['?' for _ in range(features)]]
    S = [['0' for _ in range(features)]]
    for idx, row in data.iterrows():
        instance = list(row.iloc[:-1])
        label = row.iloc[-1]
        if label == 'Yes':
            if S[0][0] == '0':
                S[0] = instance
            else:
                S[0] = [s if s == i else '?' for s, i in zip(S[0], instance)]
            G = [g for g in G if all(g[i] == '?' or g[i] == instance[i]
                                     for i in range(features))]
        else:
            new_G = []
            for g in G:
                indices = [i for i in range(features)
                           if g[i] == '?' and instance[i] != S[0][i]]
                for i in indices:
                    new_g = g.copy()
                    new_g[i] = S[0][i]
                    if new_g not in new_G:
                        new_G.append(new_g)
            G = new_G if new_G else G
        print(f"\nIteration {idx + 1}:")
        print(f"S: {S}")
        print(f"G: {G}")
    return S, G
def main():
    try:
        filename = input("Enter CSV filename: ")
        data = pd.read_csv(filename)
        print("\nData:\n", data)
        S, G = candidate_elimination(data)
        print("\nFinal Version Space:")
        print(f"Specific: {S}")
        print(f"General: {G}")
    except Exception as e:
        print(f"Error: {e}")
if __name__ == "__main__":
    main()
```

OUTPUT:

```
Enter CSV filename:  Training data.csv
```

Data:

	Sky	Temp	Humidity	Wind	Water	Forecast	Enjoy Sport
0	Sunny	Warm	Normal	Strong	Warm	Same	Yes
1	Sunny	Warm	High	Strong	Warm	Same	Yes
2	Rainy	Cold	High	Strong	Warm	Change	No
3	Sunny	Warm	High	Strong	Cool	Change	Yes

Iteration 1:
S: [['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']]
G: [['?', '?', '?', '?', '?', '?']]

Iteration 2:
S: [['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']]
G: [['?', '?', '?', '?', '?', '?']]

Iteration 3:
S: [['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']]
G: [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', 'Same']]

Iteration 4:
S: [['Sunny', 'Warm', '?', 'Strong', '?', '?']]
G: [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?']]

Final Version Space:
Specific: [['Sunny', 'Warm', '?', 'Strong', '?', '?']]
General: [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?']]

---

5. Write a program to demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

CODE:

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OrdinalEncoder
from sklearn.pipeline import Pipeline
file_path = input("Enter the path to your CSV file: ")
df = pd.read_csv(file_path)
feature_columns = list(df.columns[:-1])
target_column = df.columns[-1]
pipeline = Pipeline([
    ('encoder', ColumnTransformer([
        ('ordinal', OrdinalEncoder(), feature_columns)
    ])),
```

```
    ('classifier', DecisionTreeClassifier(criterion='entropy'))
])
X = df[feature_columns]
y = df[target_column]
pipeline.fit(X, y)
tree_rules = export_text(pipeline.named_steps['classifier'],
  feature_names=feature_columns)
print("\nDecision Tree Rules\n", tree_rules)

new_sample = pd.DataFrame([{
    feature: input(f"Enter value for {feature}: ")
    for feature in feature_columns
}])
prediction = pipeline.predict(new_sample)[0]
print("\nNew Sample:", new_sample.to_dict('records')[0])
print(f"Prediction ({target_column}): {prediction}")
```

OUTPUT:

```
Enter the path to your CSV file:  C:\Users\Nithil Surya\Desktop\AIML
  Lab\4_Decision_Tree_Based_ID3_Algo\Training data.csv

Decision Tree Rules
 |--- Sky <= 0.50
|   |--- class: No
|--- Sky >  0.50
|   |--- class: Yes

Enter value for Sky:  Sunny
Enter value for Temp:  Warm
Enter value for Humidity:  Normal
Enter value for Wind:  Strong
Enter value for Water:  Warm
Enter value for Forecast:  Same

New Sample: {'Sky': 'Sunny', 'Temp': 'Warm', 'Humidity': 'Normal', 'Wind': 'Strong',
  'Water': 'Warm', 'Forecast': 'Same'}
Prediction (Enjoy Sport): Yes
```

---

6. Build an Artificial Neural Network by implementing the Back propagation algorithm and test the same using appropriate data sets.

CODE:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error
class SimpleNN:
    def __init__(self, layers):
        self.weights = [np.random.uniform(-np.sqrt(6 / (layers[i] + layers[i+1])), np.sqrt(6
  / (layers[i] + layers[i+1])), (layers[i], layers[i+1])) for i in range(len(layers)-1)]
        self.biases = [np.zeros((1, layers[i+1])) for i in range(len(layers)-1)]
```

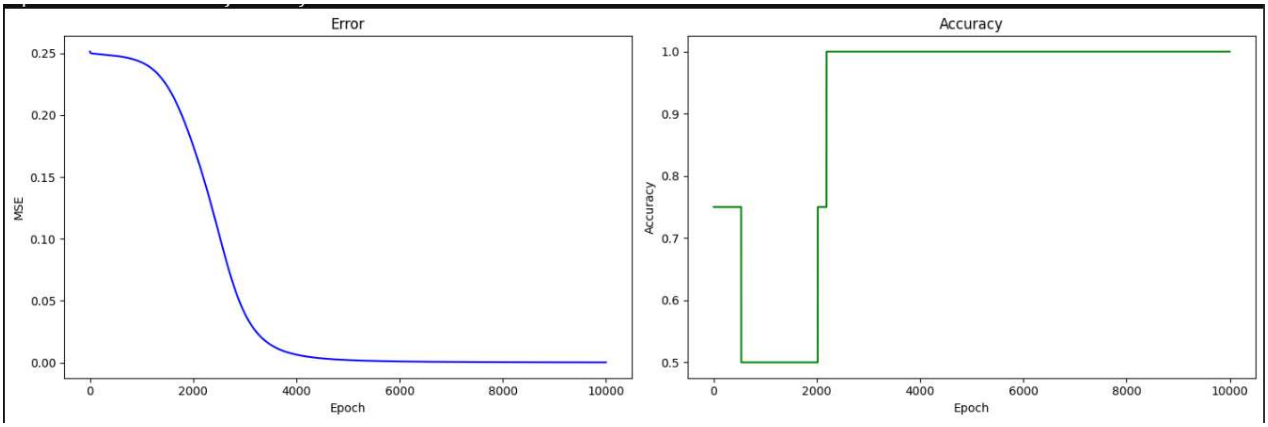
```
    def sigmoid(self, x): return 1 / (1 + np.exp(-x))
    def forward(self, X):
        self.a = [X]
        for i in range(len(self.weights)):
            X = self.sigmoid(np.dot(X, self.weights[i]) + self.biases[i])
            self.a.append(X)
        return X
    def backward(self, X, y, lr):
        m = X.shape[0]
        delta = self.a[-1] - y
        for i in range(len(self.weights)-1, -1, -1):
            dW, db = np.dot(self.a[i].T, delta), np.sum(delta, axis=0, keepdims=True)
            delta = np.dot(delta, self.weights[i].T) * self.a[i] * (1 - self.a[i])
            self.weights[i] -= lr * dW / m
            self.biases[i] -= lr * db / m
    def train(self, X, y, epochs, lr=0.01):
        errors, accuracies = [], []
        for epoch in range(epochs):
            output = self.forward(X)
            error = mean_squared_error(y, output)
            accuracy = np.mean(np.round(output) == y)
            self.backward(X, y, lr)
            errors.append(error)
            accuracies.append(accuracy)
            if (epoch + 1) % 1000 == 0:
                print(f"Epoch {epoch+1}: Error {error:.4f}, Accuracy {accuracy:.2%}")
        return errors, accuracies
    def predict(self, X): return np.round(self.forward(X))
    def plot(self, errors, accuracies):
        fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))
        ax1.plot(errors, 'b-'); ax1.set_title('Error'); ax1.set_xlabel('Epoch');
  ax1.set_ylabel('MSE')
        ax2.plot(accuracies, 'g-'); ax2.set_title('Accuracy'); ax2.set_xlabel('Epoch');
  ax2.set_ylabel('Accuracy')
        plt.tight_layout(); plt.show()
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0], [1], [1], [0]])
nn = SimpleNN([2, 4, 1])
errors, accuracies = nn.train(X, y, epochs=10000, lr=0.1)
nn.plot(errors, accuracies)
print("\nTesting results:")
for i in range(len(X)):
    pred = nn.predict(X[i:i+1])
    print(f"{X[i]} -> {pred[0][0]} -> {y[i][0]}")
print(f"\nFinal Error: {errors[-1]:.4f}, Final Accuracy: {accuracies[-1]:.2%}")
```

OUTPUT:

```
Epoch 1000: Error 0.2428, Accuracy 50.00%
Epoch 2000: Error 0.1757, Accuracy 50.00%
Epoch 3000: Error 0.0399, Accuracy 100.00%
Epoch 4000: Error 0.0064, Accuracy 100.00%
```

Epoch 5000: Error 0.0021, Accuracy 100.00%
Epoch 6000: Error 0.0009, Accuracy 100.00%
Epoch 7000: Error 0.0005, Accuracy 100.00%
Epoch 8000: Error 0.0003, Accuracy 100.00%
Epoch 9000: Error 0.0002, Accuracy 100.00%
Epoch 10000: Error 0.0002, Accuracy 100.00%
Testing results:
[0 0] -> 0.0 -> 0
[0 1] -> 1.0 -> 1
[1 0] -> 1.0 -> 1
[1 1] -> 0.0 -> 0

Final Error: 0.0002, Final Accuracy: 100.00%



7. Write a program to implement the naïve Bayesian classifier for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering few test data sets.

CODE:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report
def main():
    try:
        filename = input("Enter the CSV filename (including extension): ")
        data = pd.read_csv(filename)
        X = data.drop('Class', axis=1)
        y = data['Class']
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.4, random_state=42, stratify=y
        )
        model = GaussianNB()
        model.fit(X_train, y_train)
        predictions = model.predict(X_test)
        print("\nNaive Bayes Classifier Results:")
```

```
        print("-" * 30)
        print(f"Accuracy: {accuracy_score(y_test, predictions):.2%}")
        print("\nClassification Report:")
        print(classification_report(y_test, predictions, zero_division=0))
        print("\nSample Predictions:")
        print("-" * 30)
```

OUTPUT:

```
Enter the CSV filename (including extension):  newfeatures.csv

Naive Bayes Classifier Results:
------------------------------
Accuracy: 50.00%

Classification Report:
              precision    recall  f1-score   support

      Setosa       1.00      0.50      0.67         2
  Versicolor       0.00      0.00      0.00         1
   Virginica       0.33      1.00      0.50         1

    accuracy                           0.50         4
   macro avg       0.44      0.50      0.39         4
weighted avg       0.58      0.50      0.46         4

Sample Predictions:
------------------------------

Case 1:
Features: {'Feature1': np.float64(6.1), 'Feature2': np.float64(2.8), 'Feature3':
  np.float64(4.0), 'Feature4': np.float64(1.3)}
Predicted: Virginica
Actual: Versicolor

Case 2:
Features: {'Feature1': np.float64(5.4), 'Feature2': np.float64(3.9), 'Feature3':
  np.float64(1.7), 'Feature4': np.float64(0.4)}
Predicted: Virginica
Actual: Setosa

Case 3:
Features: {'Feature1': np.float64(5.1), 'Feature2': np.float64(3.5), 'Feature3':
  np.float64(1.4), 'Feature4': np.float64(0.2)}
Predicted: Setosa
Actual: Setosa
```

Case 4:
Features: {'Feature1': np.float64(5.9), 'Feature2': np.float64(3.0), 'Feature3': np.float64(5.1), 'Feature4': np.float64(1.8)}
Predicted: Virginica
Actual: Virginica

---

8. Apply EM algorithm to cluster a set of data stored in a .CSV file. Use the same data set for clustering using k-Means algorithm. Compare the results of these two algorithms and comment on the quality of clustering. You can add Java/Python ML library classes/API in the program.

CODE:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
import pandas as pd

file_name = input("Enter the CSV file name: ")

try:
    data = pd.read_csv(file_name)

    X = data.select_dtypes(include=[np.number]).values

    k = int(input("Enter the number of clusters (K): "))

    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X)

    labels = kmeans.labels_
    centers = kmeans.cluster_centers_

    print("\nCluster labels for each point:")
    for point, label in zip(X, labels):
        print(f"Point {point} is in Cluster {label}")

    print("\nCluster Centers:")
    for i, center in enumerate(centers):
        print(f"Cluster {i}: Center at {center}")

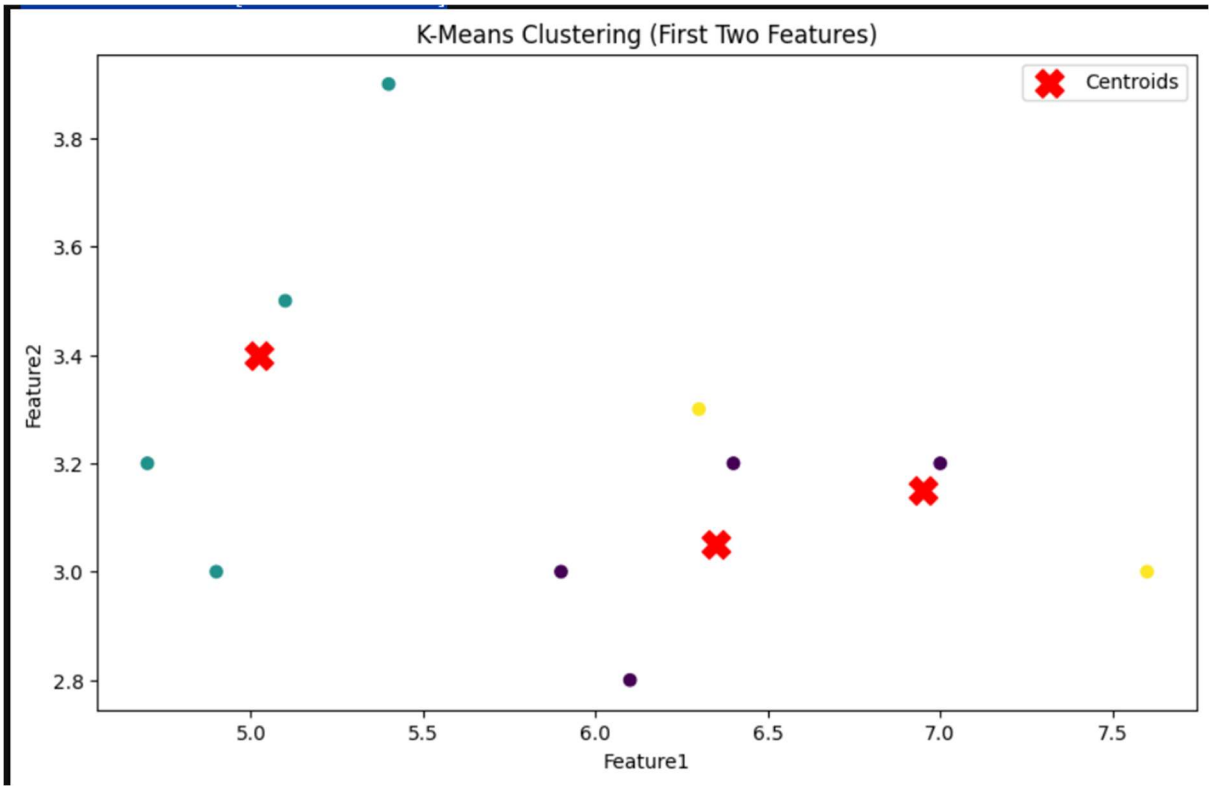
    plt.figure(figsize=(10, 6))
    plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', marker='o')
    plt.scatter(centers[:, 0], centers[:, 1], c='red', marker='X', s=200, label='Centroids')
    plt.title('K-Means Clustering (First Two Features)')
    plt.xlabel(data.select_dtypes(include=[np.number]).columns[0])
    plt.ylabel(data.select_dtypes(include=[np.number]).columns[1])
    plt.legend()
    plt.show()
```

```
except FileNotFoundError:
    print(f"Error: File '{file_name}' not found")
except pd.errors.EmptyDataError:
    print(f"Error: File '{file_name}' is empty")
except Exception as e:
    print(f"Error: {str(e)}")
```

OUTPUT:

```
Cluster labels for each point:
Point [5.1 3.5 1.4 0.2] is in Cluster 1
Point [4.9 3.  1.4 0.2] is in Cluster 1
Point [7.  3.2 4.7 1.4] is in Cluster 0
Point [6.4 3.2 4.5 1.5] is in Cluster 0
Point [5.9 3.  5.1 1.8] is in Cluster 0
Point [6.3 3.3 6.  2.5] is in Cluster 2
Point [4.7 3.2 1.3 0.2] is in Cluster 1
Point [7.6 3.  6.6 2.1] is in Cluster 2
Point [6.1 2.8 4.  1.3] is in Cluster 0
Point [5.4 3.9 1.7 0.4] is in Cluster 1

Cluster Centers:
Cluster 0: Center at [6.35  3.05  4.575 1.5  ]
Cluster 1: Center at [5.025 3.4   1.45  0.25 ]
Cluster 2: Center at [6.95 3.15 6.3  2.3 ]
```



9. Write a program to implement k-Nearest Neighbour algorithm to classify the iris data set. Print both correct and wrong predictions. Java/Python ML library classes can be used for this problem.

CODE:

```
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
import numpy as np
import matplotlib.pyplot as plt

file_path = input("Enter the path to your CSV file : ")
data = pd.read_csv(file_path)

X = data.iloc[:, :-1].values
y = data.iloc[:, -1].values

label_encoder = LabelEncoder()
y = label_encoder.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
  random_state=42)

neighbors = np.arange(1, 9)
train_accuracy = np.empty(len(neighbors))
test_accuracy = np.empty(len(neighbors))

for i, k in enumerate(neighbors):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)

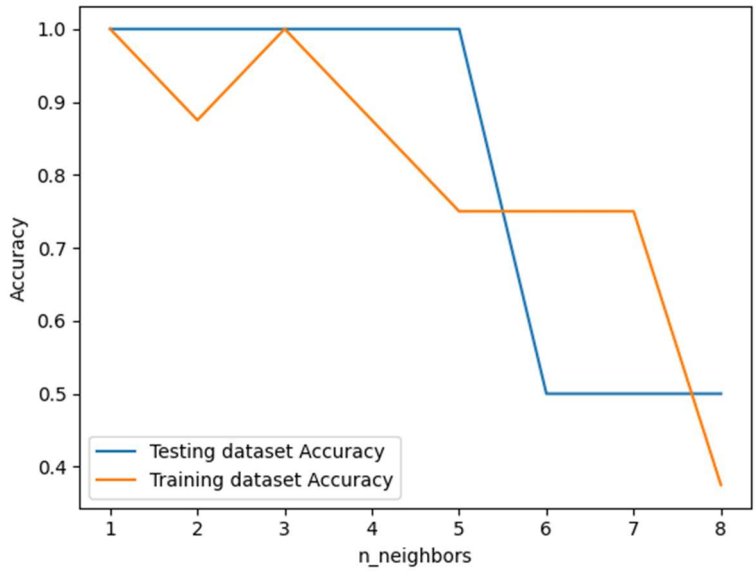
    train_accuracy[i] = knn.score(X_train, y_train)
    test_accuracy[i] = knn.score(X_test, y_test)

plt.plot(neighbors, test_accuracy, label='Testing dataset Accuracy')
plt.plot(neighbors, train_accuracy, label='Training dataset Accuracy')

plt.legend()
plt.xlabel('n_neighbors')
plt.ylabel('Accuracy')
plt.show()
```

OUTPUT:

```
Enter the path to your CSV file :  C:\Users\Nithil Surya\Desktop\AIML
  Lab\8_K_Nearest_Neighbor_Algorithm\newfeatures.csv
```



10. Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select appropriate data set for your experiment and draw graphs.

CODE:

```
import numpy as np
import matplotlib.pyplot as plt

# --- 1. Data Generation ---
np.random.seed(42)
m = 100  # Number of data points
X_data = np.linspace(-3, 3, m)
# Non-linear function: sin(x) + some noise
Y_data = np.sin(X_data) * 2 + X_data * 0.5 + np.random.normal(0, 0.4, m)

# Prepare the design matrix X (add a column of ones for the intercept)
X_design = np.vstack([np.ones(m), X_data]).T
Y_data = Y_data.reshape(-1, 1)

# --- 2. The Locally Weighted Regression Function ---
def lwr_predict(X_design, Y_data, x_query, tau):
    """
    Performs one prediction using Locally Weighted Regression.

    X_design: The full design matrix (m x 2, with intercept column)
    Y_data: The full target vector (m x 1)
    x_query: The query point (scalar)
    tau: The bandwidth parameter

    Returns: The predicted y-value for x_query.
    """
    # Create the query design vector (1, x_query)
    x_query_vec = np.array([1, x_query])
```

```
    # 1. Calculate weights W for all data points relative to x_query
    # Use only the feature values for the weight calculation (the second column of
  X_design)
    x_data_features = X_design[:, 1]

    # Squared distance
    diff = x_data_features - x_query
    squared_dist = diff ** 2

    # Gaussian weights
    weights = np.exp(-squared_dist / (2 * tau**2))
    W = np.diag(weights) # W is a diagonal matrix

    # 2. Closed-Form Solution for Theta: theta = (X^T W X)^-1 X^T W y
    XT_W = X_design.T @ W

    # Calculate (X^T W X)^-1
    try:
        XT_W_X_inv = np.linalg.inv(XT_W @ X_design)
    except np.linalg.LinAlgError:
        # Handle the case where the matrix is singular (shouldn't happen with good
  data)
        return np.nan

    # Calculate the parameters theta
    theta = XT_W_X_inv @ XT_W @ Y_data

    # 3. Prediction: y_hat = theta^T x_query_vec
    y_hat = x_query_vec @ theta

    return y_hat[0]

# --- 3. Run the Algorithm and Plot ---
def run_lwr(X_design, Y_data, tau):
    # Create a dense grid of query points for a smooth fit line
    X_test = np.linspace(X_design[:, 1].min(), X_design[:, 1].max(), 100)

    # Get predictions for all test points
    Y_pred = [lwr_predict(X_design, Y_data, x_q, tau) for x_q in X_test]

    # Plotting
    plt.figure(figsize=(10, 6))
    plt.scatter(X_design[:, 1], Y_data, label='Data Points', color='skyblue', alpha=0.6)
    plt.plot(X_test, Y_pred, label=f'LWR Fit ($\u03c4$={tau})', color='darkred',
  linewidth=3)

    plt.title(f' Locally Weighted Regression Fit with Bandwidth $\u03c4$={tau}')
    plt.xlabel('X Feature')
    plt.ylabel('Y Target')
    plt.legend()
```

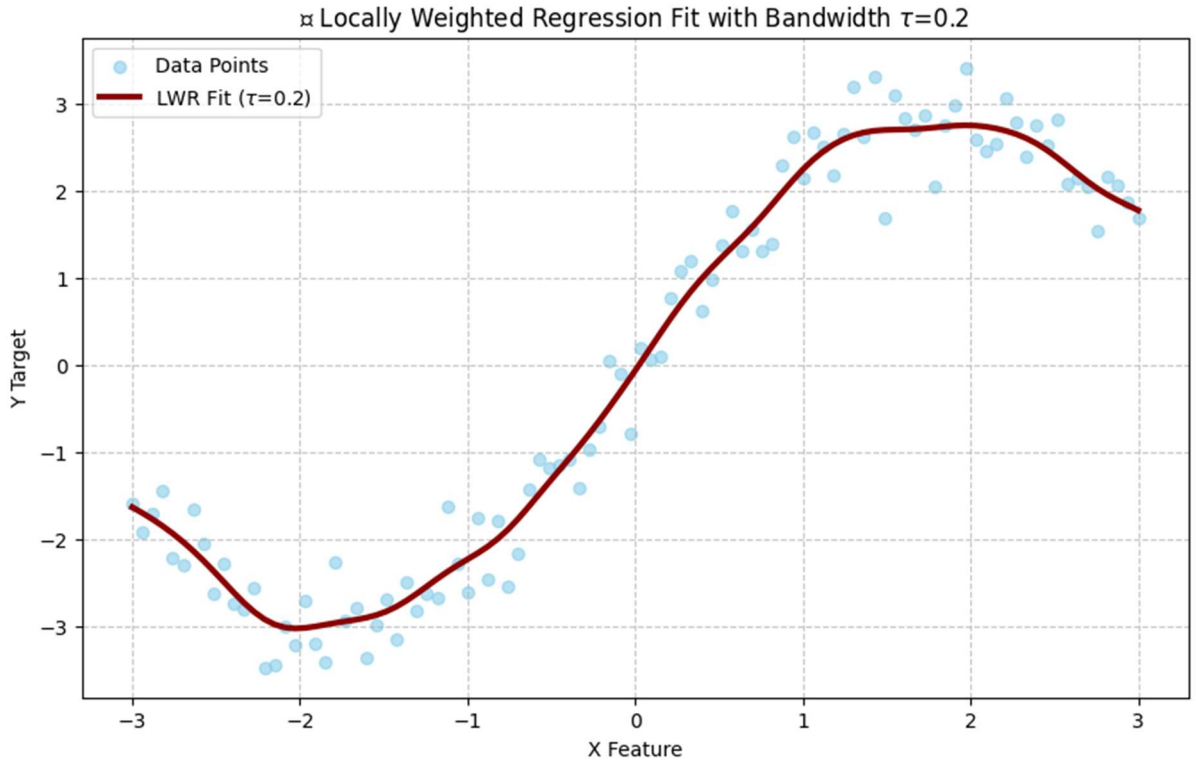
```
    plt.grid(True, linestyle='--', alpha=0.7)
    plt.show()

# Run the experiment with different tau values to show the effect of the bandwidth
print("Running LWR with a narrow bandwidth (low bias, high variance)...")
run_lwr(X_design, Y_data, tau=0.2) # Narrow band (wobbly fit)

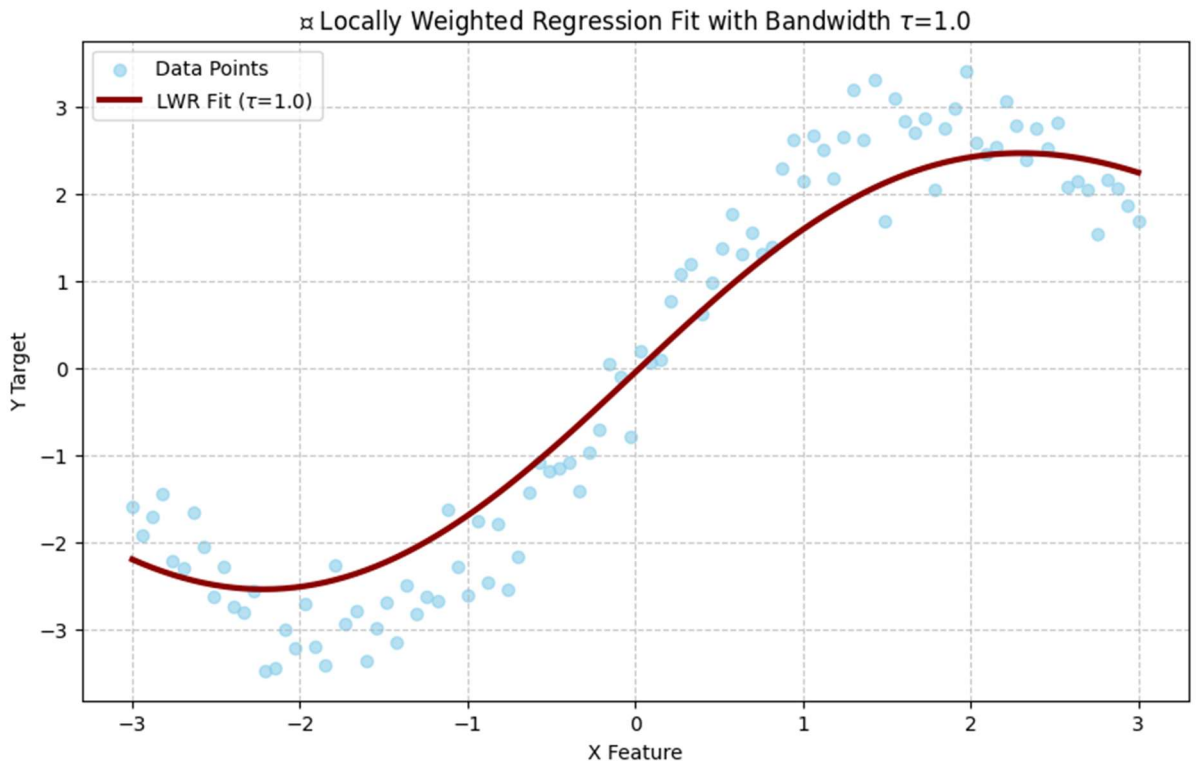
print("Running LWR with a wider bandwidth (high bias, low variance)...")
run_lwr(X_design, Y_data, tau=1.0) # Wide band (smooth fit, similar to global linear
  regression)
```

OUTPUT:

Running LWR with a narrow bandwidth (low bias, high variance)...



Running LWR with a wider bandwidth (high bias, low variance)...



---

11. Write a program to implement data preprocessing and text cleaning using python.

CODE:

```
import pandas as pd
import re
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
import nltk
def preprocess_data(df):
    """
    Performs general data preprocessing steps on a DataFrame.
    """
    for col in df.select_dtypes(include=['number']).columns:
        df[col] = df[col].fillna(df[col].median())
    # df = pd.get_dummies(df, columns=['categorical_column_name'],
  drop_first=True)
    return df
def clean_text(text):
    """
    Performs text cleaning steps.
    """
    text = text.lower()
```

```
    text = re.sub(r'[^a-zA-Z\s]', '', text)
   tokens = word_tokenize(text)
    stop_words = set(stopwords.words('english'))
    tokens = [word for word in tokens if word not in stop_words]
    lemmatizer = WordNetLemmatizer()
    tokens = [lemmatizer.lemmatize(word) for word in tokens]
    cleaned_text = ' '.join(tokens)
    return cleaned_text
if __name__ == "__main__":
    data = {'numerical_col': [10, 20, None, 40, 50],
            'categorical_col': ['A', 'B', 'A', 'C', 'B'],
            'text_data': ["This is some sample text for cleaning!",
                          "Another example sentence, with punctuation!",
                          "Stop words are removed here.",
                          "Lemmatization is also applied.",
                          "Final text cleaning example."]}
    df = pd.DataFrame(data)
    print("Original DataFrame:")
    print(df)
    df_preprocessed = preprocess_data(df.copy())
    print("\nDataFrame after general data preprocessing:")
    print(df_preprocessed)
    df_preprocessed['cleaned_text'] = df_preprocessed['text_data'].apply(clean_text)
    print("\nDataFrame after text cleaning:")
    print(df_preprocessed[['text_data', 'cleaned_text']])
```

OUTPUT:

```
Original DataFrame:
   numerical_col categorical_col                      text_data
0           10.0               A        This is some sample text for cleaning!
1           20.0               B  Another example sentence, with punctuation!
2           None               A           Stop words are removed here.
3           40.0               C         Lemmatization is also applied.
4           50.0               B          Final text cleaning example.

DataFrame after general data preprocessing:
   numerical_col categorical_col                      text_data
0           10.0               A        This is some sample text for cleaning!
1           20.0               B  Another example sentence, with punctuation!
2           30.0               A           Stop words are removed here.
3           40.0               C         Lemmatization is also applied.
4           50.0               B          Final text cleaning example.

DataFrame after text cleaning:
                          text_data                  cleaned_text
0    This is some sample text for cleaning!        sample text cleaning
```

1 Another example sentence, with punctuation! another example sentence punctuation
2 Stop words are removed here. stop word removed
3 Lemmatization is also applied. lemmatization also applied
4 Final text cleaning example. final text cleaning example

---

12. Write a program to demonstrate parts-of-speech tagging and named entity recognition.

CODE:

```
import spacy
try:
    nlp = spacy.load("en_core_web_sm")
except OSError:
    print("Error: The 'en_core_web_sm' model is not found.")
    print("Please run 'python -m spacy download en_core_web_sm' in your terminal.")
    exit()
text = "Apple Inc. announced the launch of its new product in London next month.
  Tim Cook, the CEO, expects the company to sell 10 million units."
doc = nlp(text)
print("--- Parts-of-Speech (POS) Tagging ---")
print("{:<15} {:<10} {:<10}".format("Token", "POS Tag", "Description"))
print("-" * 35)
for token in doc:
    print("{:<15} {:<10} {:<10}".format(
        token.text,
        token.pos_,
        spacy.explain(token.pos_)
    ))
print("\n--- Named Entity Recognition (NER) ---")
print("{:<20} {:<10} {:<30}".format("Entity Text", "Entity Type", "Description"))
print("-" * 60)
for ent in doc.ents:
    print("{:<20} {:<10} {:<30}".format(
        ent.text,
        ent.label_,
        spacy.explain(ent.label_)
    ))
```

OUTPUT:

Output

```
--- Parts-of-Speech (POS) Tagging ---
Token           POS Tag    Description
-----------------------------------
Apple           PROPN      proper noun
Inc.            PROPN      proper noun
```

```
announced       VERB       verb
the             DET        determiner
launch          NOUN       noun
of              ADP        adposition
its             PRON       pronoun
new             ADJ        adjective
product         NOUN       noun
in              ADP        adposition
London          PROPN      proper noun
next            ADJ        adjective
month           NOUN       noun
.               PUNCT      punctuation
Tim             PROPN      proper noun
Cook            PROPN      proper noun
,               PUNCT      punctuation
the             DET        determiner
CEO             NOUN       noun
,               PUNCT      punctuation
expects         VERB       verb
the             DET        determiner
company         NOUN       noun
to              PART       particle
sell            VERB       verb
10 million      NUM        numeral
units           NOUN       noun
.               PUNCT      punctuation

--- Named Entity Recognition (NER) ---
Entity Text          Entity Type Description
------------------------------------------------------------
Apple Inc.           ORG        Companies, agencies, institutions, etc.
London               GPE        Countries, cities, states
next month           DATE       Absolute or relative dates or periods
Tim Cook             PERSON     People, including fictional
10 million           MONEY      Monetary values, including units
```